Minicurso Git e GitHub

Material Didático Semana da Computação C3

Jhon Victor Ramos Martins

Redes Profissionais	Contato
LinkedIn: jhon-victor-ramos	**Instagram:** @jhonvictor_dev
GitHub: Jhon-Victor-Ramos	**E-mail:** jhon.vrmartins@gmail.com

Sumário

1 Git **2**

Entrando no Git . . . 2

1. Para que serve o Git . . . 2
2. Os 3 ambientes do Git . . . 2
3. Versionamento de código como linha temporal . . . 3
4. Git e GitHub: Qual a diferença? . . . 3
5. O que é um Commit . . . 4
6. Comandos essenciais . . . 4
7. Entendendo o Terminal . . . 4
8. Funcionamento das Branches . . . 5
9. Desfazendo erros . . . 5

2 GitHub **6**

10. Conectando à nuvem . . . 6
11. Trabalhando em equipe . . . 6
12. Merge Conflicts . . . 7

Bônus: Commits Semânticos **7**

1 Git

Entrando no Git

Antes de começar a usar a ferramenta, o Git exige saber quem é você. Diferente de um site ou aplicativo, isso **não é um login com senha**. É como um crachá que será usado para lhe identificar em cada alteração (*commit*) que você fizer.

Sem isso, o Git bloqueia você de salvar qualquer coisa, pois ele tem o dever de manter a auditoria perfeita de quem alterou o código e quando.

Ao abrir o terminal, rode estes dois comandos **apenas uma vez** na sua máquina. O Git vai lembrar de você para sempre:

- `git config --global user.name "Seu Nome e Sobrenome"`: Define a assinatura que aparecerá no histórico do projeto.
- `git config --global user.email "seu.email@exemplo.com"`: Define o seu e-mail de contato.
  - *Dica:* Use o mesmo e-mail que você planeja usar (ou já usa) para criar a sua conta no GitHub lá na frente!

1. Para que serve o *Git*

No mundo comum, quando queremos testar uma ideia nova ou ter um backup seguro, criamos cópias infinitas do mesmo arquivo ou pasta. Isso gera caos, perda de arquivos e, em um ambiente profissional, destrói o trabalho em equipe.

O **Git** não é um sistema de cópias. Ele é um sistema de "fotografias"lógicas. Ele monitora a evolução do seu projeto ao longo do tempo de forma inteligente.

2. Os 3 ambientes do Git

Para entender os comandos do Git, pode-se imaginar que o seu projeto é dividido em três "zonas"de trabalho invisíveis. Um arquivo viaja por essas três zonas até ser salvo definitivamente:

1. **Working Directory (Local):** É a pasta local real. Onde escrevemos os códigos, colocamos imagens etc. Quando criamos ou editamos um arquivo no nosso computador, é nessa zona que estamos mexendo. Aqui, o Git está vendo o arquivo, mas não está rastreando suas mudanças para as salvar.
2. **Staging Area (Fase intermediária):** Quando **adicionamos** um arquivo no rastreamento do Git, o mesmo é enviado para essa zona.
3. **Repository (Cofre):** É o registro histórico oficial do projeto. Quando um arquivo que está na *Staging Area* é finalizado oficialmente, suas alterações são assinadas permanentemente na linha temporal do esquema.

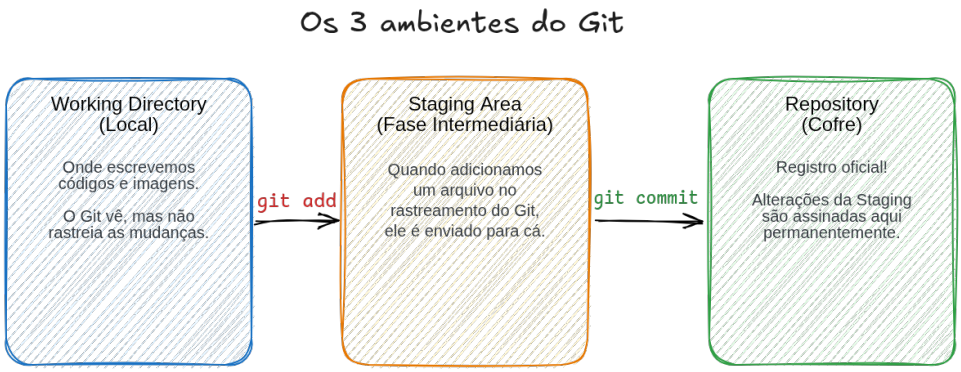

Figura 1: Ilustração das 3 zonas do Git

3. Versionamento de código como linha temporal

Para dominar o versionamento, é interessante começar a pensar nas alterações que fazemos e salvamos como **pontos nas linhas do tempo do projeto**.

- **Linha do tempo (`Log`):** Um registro imutável e auditável de quem fez o quê, quando e por quê.
- **Viagem entre os pontos (`Restore` / `Revert`):** A capacidade de visitar as versões passadas do seu projeto e, caso algo dê errado no presente, trazer o código funcional de volta ao ponto atual.
- **Bifurcações na linha do tempo (`Branches`):** É o que permite criar uma nova linha do tempo, partindo da atual, criar alterações no código e, caso a ideia seja aprovada, fundir (fazer um `merge`) a linha temporal alternativa com a principal.

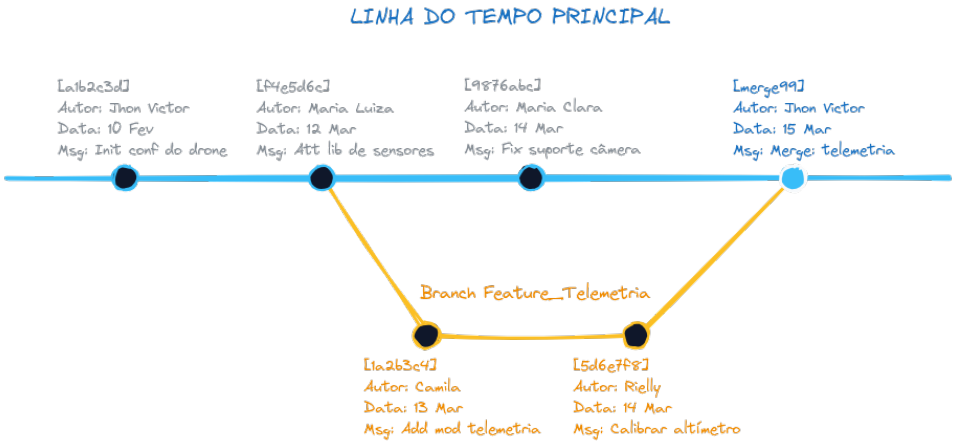

Figura 2: Demonstração de como as branches funcionam

4. Git e GitHub: Qual a diferença?

É comum ouvir de quem está iniciando a confusão entre Git e GitHub, acreditando ser a mesma coisa. Entretanto, **não** é!

- **Git (Local):** É o software de linha de comando que roda *offline* no seu computador no terminal.
- **GitHub (Nuvem):** É um serviço de hospedagem na nuvem. É a plataforma que pega a sua ação local e a coloca na internet, permitindo que recrutadores vejam seu portfólio e que outros programadores colaborem com você.

5. O que é um *Commit*

Quando não queremos perder algo no nosso computador (como um arquivo `.docx` que estamos escrevendo há um bom tempo), apertamos `Ctrl+S` para salvar. De forma análoga, um *commit* no Git salva as alterações feitas nos arquivos **dentro do diretório** (uma pasta) na qual ele, o Git, foi inicializado.

6. Comandos essenciais

O ciclo de vida de um arquivo no Git segue sempre a mesma lógica.

- `git init`: Inicia o monitoramento do Git na pasta atual.
- `git status`: Mostra o que foi alterado, o que é novo e o que está pronto para ser salvo.
- `git add .`: Prepara as alterações para o próximo *commit*. Ou seja, envia os arquivos para a segunda área de trabalho *Staging Area*.
  - *A flag `.` (ponto) significa "adicione todos os arquivos do diretório atual".*
- `git commit -m "mensagem"`: Registra o estado atual dos arquivos adicionados pelo *commit* e os salva na linha do tempo.
  - *A flag `-m` (message) permite que você escreva o título do commit direto no terminal, sem precisar abrir um editor de texto.*
- `git log --oneline`: Mostra o histórico dos commits no tempo de forma resumida.

7. Entendendo o Terminal

O terminal não cospe textos aleatórios, ele gera relatórios precisos. Quando você faz um commit, a saída será algo parecido com isso:

```
[main 29e3f09] feat: adicionei uma nova linha
1 file changed, 1 insertion(+)
```

Embora possa parecer confuso à primeira vista, é simples! Significa:

- `main`: É a **branch** onde você está no momento do commit.

- `29e3f09`: É o **Hash Curto**. Um identificador **único** como um CPF, por exemplo daquela versão do arquivo.
- `feat: ...`: É a **Mensagem** do commit. Onde explica o que foi feito nessa alteração.
- `1 file changed...`: O resumo do que o Git fez, quantos arquivos mudaram e quantas linhas foram adicionadas ou removidas.

A estrutura padrão de um aviso de salvamento é sempre esta:

```
[nomeBranch hashCurto] commitMessage
resumoAlterações
```

8. Funcionamento das Branches

- `git branch`: Lista todas as linhas do tempo que existem no seu projeto.
- `git switch -c <nome>`: O comando `switch` transiciona entre branches.
  - *A flag `-c` (create) cria a nova branch e já te teletransporta para ela de uma vez só.*
- `git merge <nome>`: Puxa as alterações de uma linha do tempo paralela (a que está no lugar de `<nome>`) para dentro da linha do tempo atual, que você está no momento.

9. Desfazendo erros

- `git restore --staged <arquivo>`: Tira um arquivo da staging area (desfaz um `git add` feito por engano), mas **não apaga** o que foi digitado.
- `git revert <hash>`: Ele não apaga o passado. Ele cria um *novo commit* que anula as alterações de um erro anterior ideal para trabalho em equipe.
- `git reset --hard <hash>`: Apaga o commit e volta para a versão anterior dos arquivos.
  - *A flag `--hard` é usada para jogar no lixo absolutamente todas as modificações que vieram depois daquele ponto, é perigoso se o código já estiver na Nuvem.*

2 GitHub

10. Conectando à nuvem

Como já dito anteriormente, o GitHub é um serviço de hospedagem na nuvem. Com ele, seu projeto ganha uma URL e visibilidade caso deixe o repositório público para recrutadores. Para que seja possível a transitação do código local até aqui, uma ponte deve ser construída.

- `git remote add origin <link_do_repositorio>`: Conecta o repositório local a um endereço específico do GitHub. Esse comando acontece somente uma vez por projeto.
  - **Entendendo a estrutura do comando:**
    * `git remote add`: É a informação de que, para o diretório local, desejamos adicionar um endereço remoto.
    * `origin`: É um apelido que damos para o servidor. Ao invés de colar o link do repositório, simplesmente escrevemos origin. É uma convenção, mas pode digitar qualquer nome (poderia chamar até de `meu_cofre_legal`).
    * `https://...`: É o link do repositório. O destino!
- `git push -u origin main`: Ele pega tudo que está no seu **Repository (Cofre)** e faz o upload para o GitHub.
  - **Entendendo a estrutura do comando:**
    * `git push`: **Enviar** o código.
    * `-u` (*upstream*): Define o caminho padrão. Serve para, quando formos usar esse comando novamente, precisemos digitar somente `git push` e o Git saiba para onde o enviar.
    * `origin`: É a forma que o Git chama a URL do repositório no GitHub. É mais rápido, prático e fácil escrever `origin` do que colar uma URL sempre que digitar esse comando.
    * `main`: A branch para a qual nosso código está sendo enviado.

11. Trabalhando em equipe

A vantagem dos códigos na nuvem além de ser uma forma de backup é trabalhar com outras pessoas no mesmo projeto sem precisar encaminhar arquivos pelo e-mail ou outra aplicação de comunicação.

- `git clone <link>`: Baixa um projeto inteiro da internet para o seu computador. O projeto já vem configurado e com todo o histórico.
- `git pull`: Se um colega de equipe realizar uma alteração no código e enviar para o GitHub, o comando `pull` faz com que puxemos essas alterações para a máquina local e tudo se mantenha sincronizado.

- **Entendendo a estrutura do comando:**
  * `git pull`: Serve para **puxar** o código. Faz duas coisas ao mesmo tempo: baixa as diferenças entre as versões e já tenta mesclá-las (realizar um *merge*).
  * `origin`: De onde estamos tentando puxar (está usando o apelido que demos antes).
  * `main`: A branch que queremos sincronizar.

12. Merge Conflicts

Quando duas pessoas alteram a mesma linha, o Git não sabe qual versão do código manter e gera um conflito na fusão das duas versões. O Git pausa o processo e marca o seu arquivo com alguns símbolos. Quando o conflito acontece, o arquivo fica assim:

```
<<<<<<< HEAD
Seu código (O que você mudou)
=======
Código do colega (O que está no GitHub)
>>>>>>> 71e904d
```

- `<<<<<<< HEAD` O que está abaixo disso é o que está na **sua** máquina agora.
- `=======`: O divisor entre as versões.
- `>>>>>>>`: O código que veio da nuvem ou de outra branch.

O seu trabalho como desenvolvedor é simplesmente abrir esse arquivo, ler as duas versões, escolher qual é a correta (ou mesclar as duas manualmente), apagar as marcações do Git e fazer um novo ciclo de `git add .` e `git commit`.

Bônus: Commits Semânticos

É comum, quando não entendemos o peso que um commit carrega, realizá-los com mensagens vagas e não organizadas. Contudo, bons repositórios possuem mensagens de commit limpas e organizadas. Afinal, os commits são a história desse repositório. Para manter a organização, o mercado adota um padrão chamado **Conventional Commits**. Onde inicia-se a mensagem com uma palavra-chave (em inglês) que categoriza o tipo de alteração feita:

- `feat:` (*Feature* / Funcionalidade) → Quando você adiciona algo novo.
  - `feat: adiciona sistema de login`
- `fix:` (*Fix* / Correção) → Quando você conserta um bug ou erro no código.
  - `fix: corrige botão de envio que não funcionava`

- `docs:` (*Documentation* / Documentação) → Quando você altera apenas arquivos de texto, como o README ou os comentários.
  - `docs: atualiza instruções de instalação`
- `refactor:` (*Refatoração*) → Quando você melhora o código interno, mas sem alterar o que ele faz visualmente para o usuário.
  - `refactor: simplifica a lógica do cálculo de média`

Dica para controle de versionamento e commit

Se você precisa usar a palavra "E"na mensagem (ex: *"feat: fiz o botão de login E corrigi a tela inicial"*), o seu commit está grande demais!
O ideal é dividir isso em dois commits menores e separados: um `feat` e um `fix`.